

NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE



PROJECT 2

LINEAR REGRESSION

COURSE: APPLIED MATHEMATICS AND STATISTIC

INSTRUCTORS

Vũ Quốc Hoàng

Phan Thị Phương Uyên Nguyễn Ngọc Toàn

STUDENT

22127073 – Nguyễn Văn Đức

Class: 22CLC08

August 14th, 2024

Table of contents

I. Introduction	3
1. Requirement	3
2. Noted	3
II. Implementation	3
1. Libraries and functions explanation	4
a) Libraries	4
b) Functions	4
2. Code explanation	5
a) EDA	5
b) Model design and testing	9
III. Reference	15

I. Introduction

- The goal of this project is to find the relationship between multiple elements and Academic Student Performance Index. The element could be the number of hours used to study/research, do student take extracurricular activities, ...

1. Requirement

- You are required to:

- Perform EDA on given data
- Build model to predict Performance Index using linear regression
 - Build a model with all 5 features
 - Build a model with one best feature. This best feature is found using k-fold Cross Validation (minimum is 5 folds).
 - Build m models and use k-fold Cross Validation to find the best one

2. Noted

- Reported value must be MAE (mean absolute error)

- You are allowed to use any library, but you must include explanations for its function purpose and detailed information.

II. Implementation

1. Libraries and functions explanation

a) Libraries

- These following libraries are used in this project:

- Pandas: used to read csv file and hold data
- Numpy: used to perform matrix manipulation
- Matplotlib: used to help seaborn with visualization
- Seaborn: used to visualize data via multiple forms of graph
- Sklearn: used to perform k-fold cross validation

b) Functions

- *class OLSLinearRegression*[\[1\]](#): linear regression handler

- *fit(self, X, y)*: train data('X') and train data('y') are fed into this function and optimal parameters are generated.
- *get_params()*: return parameters as a matrix (column vector).
- *predict(self, X)*: calculate the predict values according to test data('X') and return it as a matrix.

- *preprocess(x)*: add a column of ones to matrix 'x'.

- *MAE(y, y_hat)*: calculate the mean absolute error.

- *sklearn.model_selection.KFold(n_splits)*: return a K-fold cross-validator with n_splits.

- `sklearn.model_selection.KFold.split(X)`: split training data('X') into `n_splits` and return list of indices for both new Train data and new test Train data.

2. Code explanation

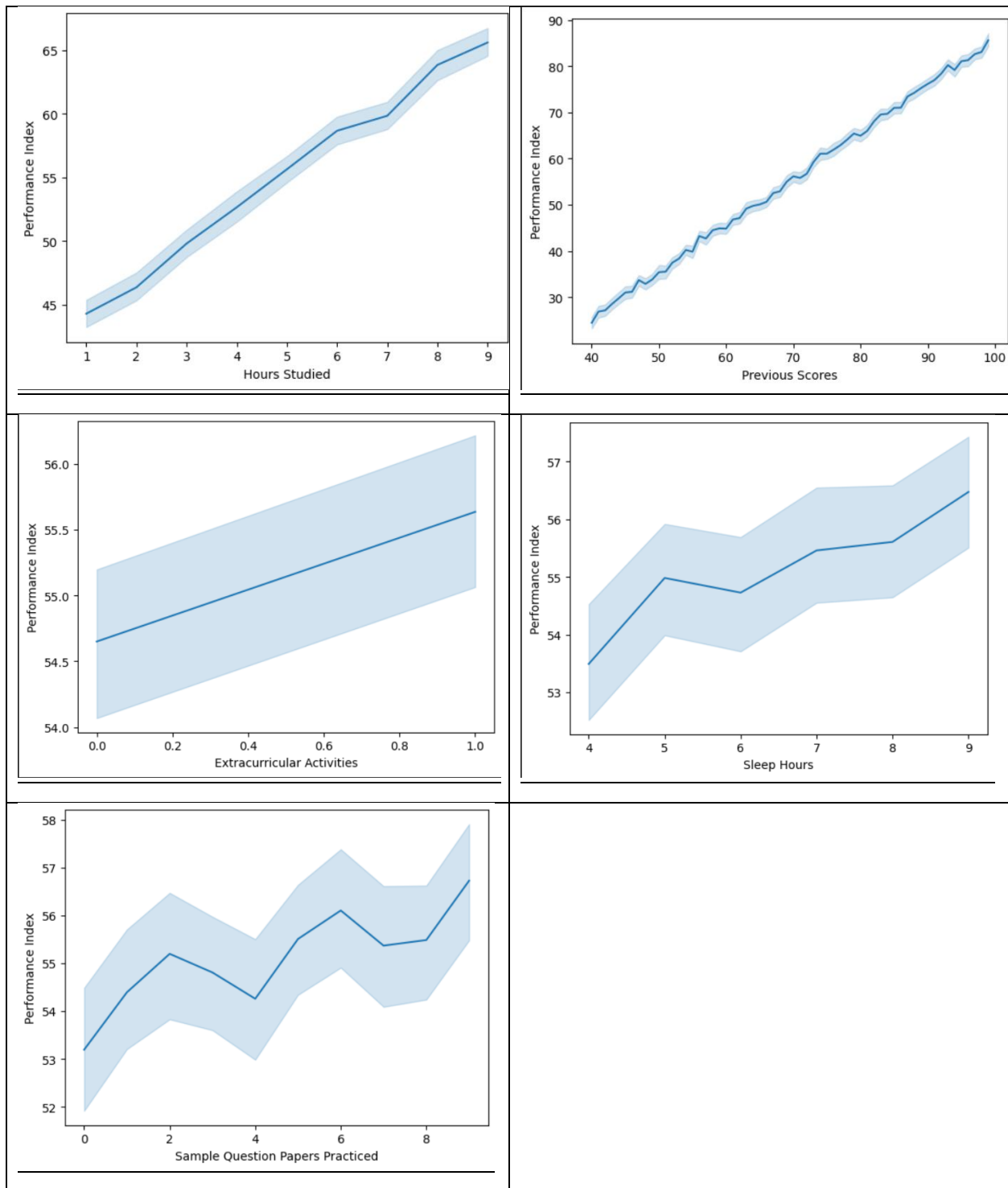
a) EDA

- First, I look up the data general information using `train.info()`.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9000 entries, 0 to 8999
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Hours Studied                        9000 non-null   int64
1   Previous Scores                      9000 non-null   int64
2   Extracurricular Activities          9000 non-null   int64
3   Sleep Hours                         9000 non-null   int64
4   Sample Question Papers Practiced    9000 non-null   int64
5   Performance Index                   9000 non-null   float64
dtypes: float64(1), int64(5)
memory usage: 422.0 KB
```

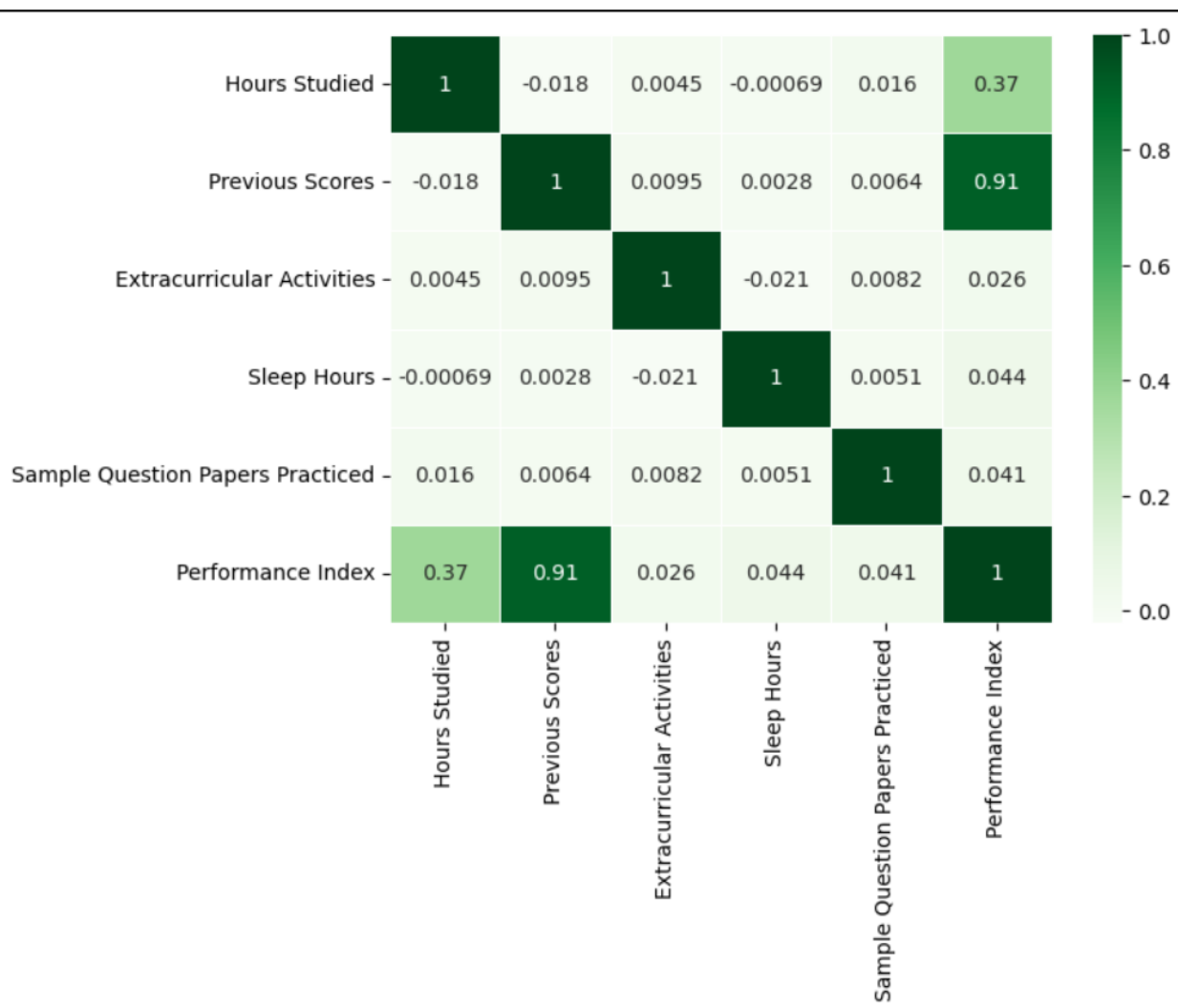
* There are 9000 lines of data, in which no null data are found, and most of their data type are int64 whilst 'Performance Index' is float64.

- Next I call `sns.lineplot(data, x, y)`: to see the relationship between all features and the target feature.



* We can see a pattern that for every feature the higher the value the better the student will perform. But notice how abnormal the line is in the last picture; this might be a sign of data overfitting.

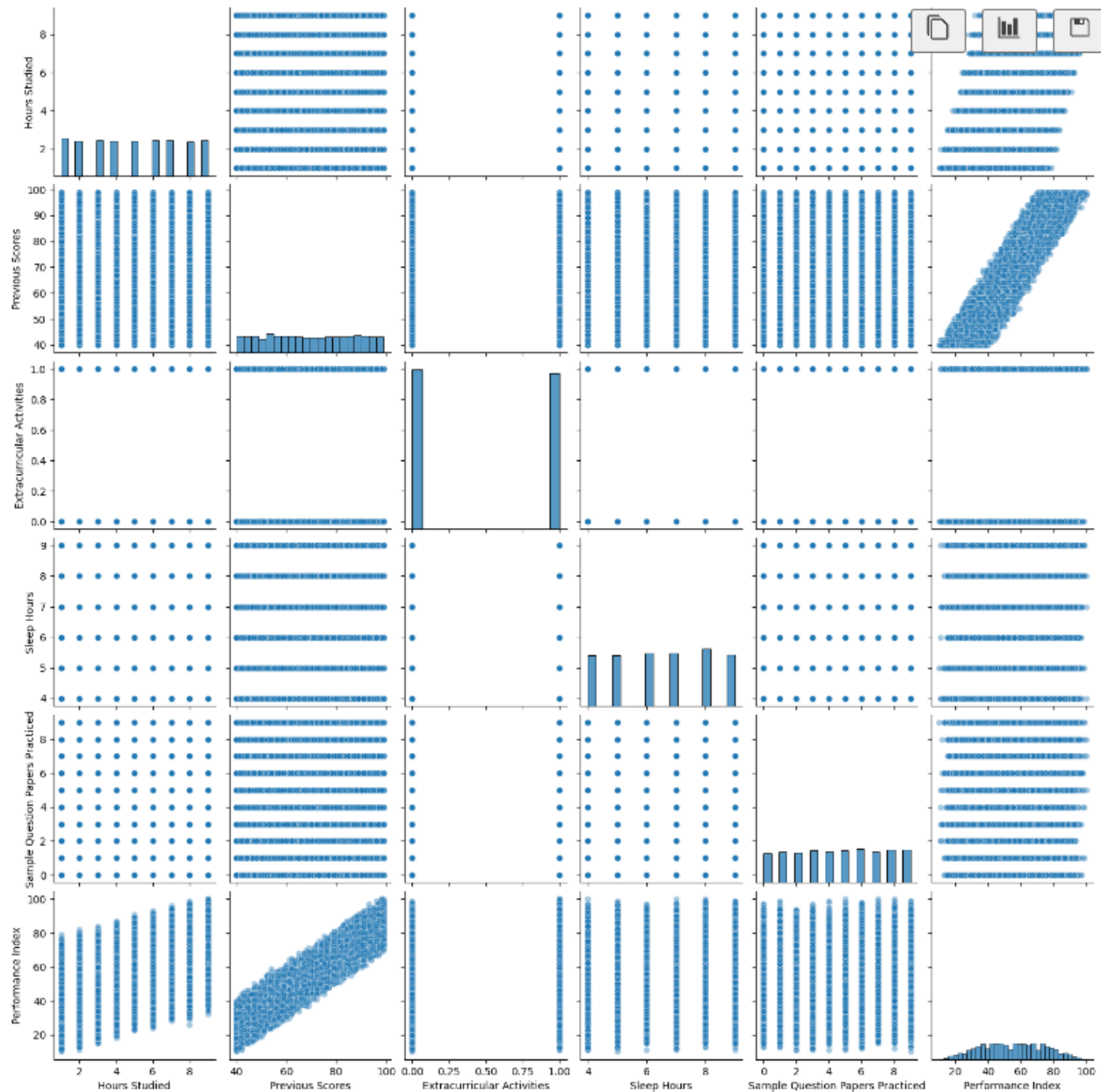
- After that I check the correlation coefficient value to reveal more patterns.[\[3\]](#)



* Looking at the heatmap we can see 'Previous Scores' feature have an insanely high correlation value. From here we can assume the 'Previous Scores' is going to be the best feature that dictates the linear regression model. Although 'Hours Studied' has the

second highest correlation value but its only considered little positivity. The rest have almost no effect on the model.

- Lastly, I draw a scatter plot to have an overview of the data:



* As I have stated we can see how impactful ‘Previous Scores’ feature is comparing to other feature such as ‘Sleep Hours’, which we can not tell anything from it.

b) Model design and testing

* Model with all features:

- Change ‘X_train’, ‘y_train’, ‘X_test’ and ‘y_test’ to numpy via *pandas.Series.to_numpy()*. For ‘y_train’ and ‘y_test’ I reshape them into column matrix for later manipulation.
- Add intercept using *preprocess(x)* for both ‘X_train’ and ‘X_test’
- Create model with *class OLSLinearRegression* then put ‘X_train’ and ‘y_train’ into *model.fit()*. Call *model.get_params()* to get parameters for the formula
- Get predict values ‘y_predict’ through *model.predict(X_test)*.
- Call *MAE(y_test, y_predict)*

=> I got 1.596 as the mean absolute error for this model.

* Model with one best features:

- K-fold cross validation:

- Started out I cloned the train data set, shuffle the clone then split it into ‘X_train_shuffle’ and ‘y_train_shuffle’.
- Create k-fold cross validator and split ‘X_train_shuffle’ into ‘k’ pair indices, each of the pair is stored in a list ‘folds’.

- Go through every feature. For each feature, it will go through every fold in ‘folds’:
 - Retrieve `trainX_BF`, `trainY_BF`, `testX_BF`, `testY_BF` from `X_train_shuffle` and `y_train_shuffle` using the indices in fold.
 - Convert them to numpy then reshape the y data set to column vector and add a column of ones to x data set
 - Create model with `class OLSLinearRegression().fit()` and get `y_pred` from `model.predict(testX_BF)`.
 - Call `MAE(testY_BF, y_pred)`
- After each fold its mae value is append to a list ‘list_mae’ then when it is done with all the fold the min mae value from ‘list_mae’ and its feature name is appended to ‘list_min_mae’.
- After the loop, I get the lowest value from ‘list_min_mae’ and print out the best feature name. Then came the mae table showing all features mae value from ‘list_min_mae’.

	Feature	MAE
0	Hours Studied	15.293
1	Previous Scores	6.456
2	Extracurricular Activities	15.744
3	Sleep Hours	15.753
4	Sample Question Papers Practiced	15.730

- Model training with best feature:

- Store 'X_train_best' with the best feature data from 'X_train' and 'X_test_best' with the best feature data from 'X_test'. Reshape it into column matrix.
- Add intercept using *preprocess(x)* for both 'X_train_best' and 'X_test_best'
- Create `best_feature_model` with *class OLSLinearRegression* then put 'X_train_best' and 'y_train' into *model.fit()*. Call *model.get_params()* to get parameters for the formula
- Get predict values 'y_predict' through *model.predict(X_test_best)*.
- Call `MAE(y_test, y_predict)`

=> I got 6.542 as the mean absolute error for this model.

* Best custom model:

- For this section, I decided to perform some feature engineering on 'Sample Question Papers Practiced' because of its abnormal line plot in EDA. For convenience I will call this feature 'Sample'.

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index	Sample Question Papers Practiced PO1/2	Sample Question Papers Practiced PO1/3	Sample Question Papers Practiced PO2	Sample Question Papers Practiced PO3
Hours Studied	1.000000	-0.018463	0.004511	-0.000694	0.015852	0.369148	0.021189	0.023628	0.011195	0.009208
Previous Scores	-0.018463	1.000000	0.009533	0.002802	0.006417	0.914775	0.007321	0.008008	0.006689	0.007393
Extracurricular Activities	0.004511	0.009533	1.000000	-0.020773	0.008199	0.025637	0.012571	0.015333	0.005737	0.004851
Sleep Hours	-0.000694	0.002802	-0.020773	1.000000	0.005054	0.043980	0.003164	0.001727	0.005276	0.004337
Sample Question Papers Practiced	0.015852	0.006417	0.008199	0.005054	1.000000	0.041088	0.957847	0.894517	0.963104	0.909312
Performance Index	0.369148	0.914775	0.025637	0.043980	0.041088	1.000000	0.042499	0.042048	0.038432	0.036525
Sample Question Papers Practiced PO1/2	0.021189	0.007321	0.012571	0.003164	0.957847	0.042499	1.000000	0.984492	0.856442	0.779471
Sample Question Papers Practiced PO1/3	0.023628	0.008008	0.015333	0.001727	0.894517	0.042048	0.984492	1.000000	0.765635	0.683204
Sample Question Papers Practiced PO2	0.011195	0.006689	0.005737	0.005276	0.963104	0.038432	0.856442	0.765635	1.000000	0.986023
Sample Question Papers Practiced PO3	0.009208	0.007393	0.004851	0.004337	0.909312	0.036525	0.779471	0.683204	0.986023	1.000000

- Here I created 4 new features ‘Sample’^{1/2}, ‘Sample’^{1/3}, ‘Sample’² and ‘Sample’³.

- Model creation explanation:

- Looking at the picture I saw an improvement of 0.01 in ‘Sample’^{1/2} so I decided to use it as a model for this section. This feature is called ‘Sample Question Papers Practiced PO1/2’[\[4\]](#)
- Since the first model doesn’t have such a high correlation value, I thought to group all 3 features with the lowest correlation value as a model to go up against the first one.
- For the last one, I found a research online[\[2\]](#) on how ‘Hours Studied’ and ‘Sleep Hours’ when both are high will result in good performance from student, so I build a model with 2 features.

- K-fold cross validation:

- Started out I cloned the train data set, shuffle the clone then split it into 'X_train_shuffle' and 'y_train_shuffle'.
- Create k-fold cross validator and split 'X_train_shuffle' into 'k' pair indices, each of the pair is stored in a list 'folds'.
- Go through every model from 'model_list'. For each model, it will go through every fold in 'folds':
 - Retrieve trainX_BM, trainY_BM, testX_BM, testY_BM from model and y_train_shuffle using the indices in fold.
 - Convert them to numpy then reshape the y data set to column vector and add a column of ones to x data set
 - Create model with `class OLSLinearRegression().fit()` and get y_pred from `model.predict(testX_BM)`.
 - Call `MAE(testY_BM, y_pred)`
- After each fold its mae value is append to a list 'list_mae' then when it is done with all the fold the min mae value from 'list_mae' and its feature name is appended to 'list_min_mae'.
- After the loop, I get the lowest value from 'list_min_mae' and print out the best feature name. Then came the mae table showing all features mae value from 'list_min_mae'.

The best feature is: Hours Studied and Sleep Hours

	Feature	MAE
0	Extracurricular Activities and Sleep Hours and...	16.045
1	Hours Studied and Sleep Hours	15.193
2	Sample Question Papers Practiced PO1/2	16.077

- Train with best model:

- Store 'X_train_BM' with the best feature data from 'X_train' and 'X_test_BM' with the best feature data from 'X_test'.
- Add intercept using *preprocess(x)* for both 'X_train_BM' and 'X_test_BM'.
- Create `my_best_model` with *class OLSLinearRegression* then put 'X_train_BM' and 'y_train' into *model.fit()*. Call *model.get_params()* to get parameters for the formula.
- Get predict values 'y_predict' through *model.predict(X_test_BM)*.
- Call *MAE(y_test, y_predict)*.

=> I got 15.202 as the mean absolute error for this model.

c) Conclusion

STT	Model	MAE
1	Model with all 5 features	1.596
2	K-Fold Model with 1 feature ('Hours Studied')	15.293
3	K-Fold Model with 1 feature ('Previous Scores')	6.456
4	K-Fold Model with 1 feature ('Extracurricular Activities')	15.744
5	K-Fold Model with 1 feature ('Sleep Hours')	15.753
6	K-Fold Model with 1 feature ('Sample Question Papers Practiced')	15.730
7	Model with 1 best feature ('Previous Scores')	6.542
8	K-Fold Model with 3 features ('Extracurricular Activities', 'Sleep Hours', 'Sample Question Papers Practiced')	16.045

9	K-Fold Model with 2 features ('Hours Studied', 'Sleep Hours')	15.193
10	K-Fold Model with 1 modified feature ('Sample Question Papers Practiced' ^{1/2})	16.077
11	Model with 2 features ('Hours Studied', 'Sleep Hours')	15.202

- From this table we can see how important 'Previous Scores' is to this linear regression model, it appears in both two lowest mae model.

III. Reference

[1]: Lab4: OLS Linear Regression from Miss Uyen, Access date: 08/08/2024

[2]: <https://journal.binus.ac.id/index.php/BECOSS/article/view/8632>, PDF file, Access date: 10/07/2024

[3]: <https://github.com/NgocTien0110/Applied-Mathematics-and-Statistics/tree/main>, Access date: 08/08/2024

[4]: <https://jdnguyen91.medium.com/feature-transformation-for-linear-regressions-1151ed1fdade>, Access date: 10/08/2024